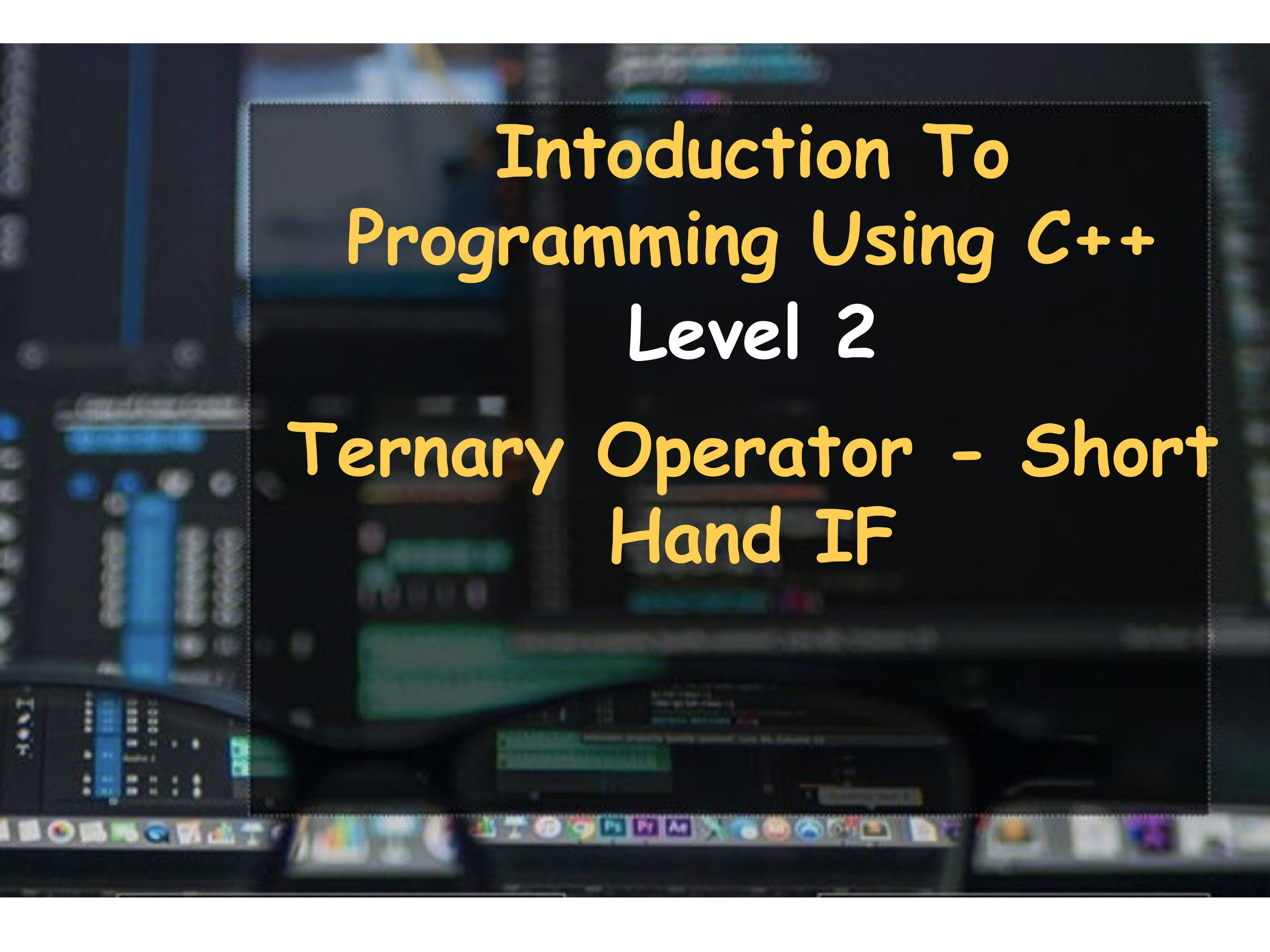


The background of the slide is a blurred image of a computer monitor. On the left side, a calculator is visible, showing some numbers and symbols. The main part of the monitor displays what appears to be a code editor or a terminal window with some text and colored highlights. The overall image is dark and out of focus, serving as a backdrop for the text.

Introduction To Programming Using C++ Level 2

Ternary Operator - Short Hand IF



Introduction To Programming Using C++

Level 2

Ternary Operator - Short Hand IF

Definition

- ◆ The Ternary Operator is a short form of an `if-else` statement.
- ◆ It is called "Ternary" because it uses three parts: a Condition, an expression for True, and an expression for False.

Explanation

■ The Ternary Operator uses the following syntax:

Condition ? expressionIfTrue : expressionIfFalse;

■ The `?` separates the Condition from the expression that is executed when the Condition is True.

■ The `:` separates the True expression from the expression that is executed when the Condition is False.

■ The result of the Ternary Operator can be assigned directly to a Variable.

Procedures

1. Write the Condition.
2. Add the `?` operator.
3. Write the expression that should be used if the Condition is True.
4. Add the `:` operator.
5. Write the expression that should be used if the Condition is False.
6. Store or use the resulting value.

💡 Example

◆ Suppose we have:

```
int Mark = 50;
```

```
string Result;
```

```
Result = (Mark >= 50) ? "Pass" : "Fail";
```

◆ The Condition is:

```
Mark >= 50
```

◆ If the Condition is True, ` "Pass" ` is assigned to ` Result `.

◆ If the Condition is False, ` "Fail" ` is assigned to ` Result `.

➡ Result

Because ` Mark ` is 50, the Condition ` Mark >= 50 ` is True.

Therefore, ` Result ` will contain ` "Pass" `.

Note

- ◆ The Ternary Operator is useful when the `if-else` logic is simple and produces a value.
- ◆ It can make short conditions more concise.

⊘ Warning

◆ Avoid using deeply nested or complicated Ternary Operators because they can make the code difficult to read.



Comparison

▨ Ternary Operator

- ✓ Shorter syntax for simple `if-else` conditions.
- ✓ Can directly return or assign a value.
- ✓ Useful for simple conditions.

▤ if-else

- ✓ More suitable for complex logic.
- ✓ Easier to read when multiple statements are required.
- ✓ Allows different blocks of code to be executed.

✦✦ Summary

- ✓ The Ternary Operator is a short form of `if-else`.
- ✓ Its syntax is `Condition ? expressionIfTrue : expressionIfFalse`.
- ✓ It is useful for simple conditions that produce a value.

Thank you 😊